

The Turing Discovery Gap Conjecture

M. Drake Stapleton

October 2, 2026

How much explanatory gain can exist beyond the reach of efficient discovery?

Revision 9, 2026-10-02. In this revision I restate the Claim A uniform scoring bound operationally: the profile fixes one polynomial step budget for the whole evaluation, the scorer executes submitted code under that budget and rejects timeouts or failed reconstruction, and the key-holding reference program fits within the budget. I also mark the CEXP-1 measurement profile and the appendix reference as draft preregistration status, not frozen.

Earlier history: Revision 8, 2026-10-02. In this revision I narrow CEXP-1 to equal candidate-evaluation allowances, with physical costs reported as outcomes rather than assumed equal; retitle the CEXP-1 text as a draft preregistration specification; state the Claim A uniform scoring bound as a scorer requirement; and update the cited paper versions to formal v1.12 and companion v1.6.

Central claim

I claim there exist worlds governed by compact, efficiently executable rules, whose explanations yield enormous gains in Turings, yet remain beyond the reliable reach of every computationally efficient learner.

Knowing a rule and discovering that rule are different computational tasks. A short program may explain a vast collection of observations while offering no efficient path from those observations to an effective explanation.

The explanation can be small while the distance to finding it is vast.

The Turing as a unit

One Turing represents one bit of net held-out description-length gain, after accounting for the description cost of the model, in the minimum-description-length sense [10].

An evaluation record consists of an input location and its outcome. Let X denote the input locations of the evaluation set and D the outcomes at those locations. The input locations are supplied to both models: they are the questions, not the answers. Let W denote the hidden generating program that produced the outcomes; W is never part of the declared background. For a candidate model M , a fixed baseline B , the evaluation outcomes D , the declared shared background G (here: the input list X and the scoring conventions), and the frozen measurement profile P , define:

$$T(M; B, D, G, P) = [L(B | G) + L(D | B, G)] - [L(M | G) + L(D | M, G)].$$

Here, L denotes description length in bits under a specified encoding.

One accounting rule governs everything a decoder may use: anything available to the decoder beyond X is part of the model and is charged in its description length. If a model decodes with the help of a retained training history, that history is written into the model and charged in $L(M)$. No information reaches the decoder uncharged, for either model, in any claim below.

A model earns Turings by describing fresh outcomes more economically than the baseline, after paying for its own complexity. Memorizing the evaluation data does not constitute discovery.

Two further rules accompany the unit:

- **Report the density, not only the total.** Alongside total Turings, report Turings per evaluated bit. I call this the gain density: total gain normalized by evaluation size. Normalization removes the raw growth of the total as the evaluation set grows. The density is not strictly size-independent: a model's description cost is paid once, so it is amortized over more evaluated bits as the set grows, and the density can change with evaluation size for that reason. The density does not by itself determine whether a gain was earned or bought; it states how much gain each evaluated bit carried.
- **Coding must be efficient.** The encoding and decoding procedures associated with a model must run in time polynomial in the size of the model and the data. Without this rule, a description can conceal enormous computation and description length stops measuring explanation.

Proposed mathematical form

I propose a family of learning problems, indexed by a size parameter n , with the following properties:

1. **Compact explanation.** Each hidden world has a generating program with description length $O(n)$.
2. **Efficient execution.** Given that program, outcomes can be predicted in time polynomial in n .
3. **Large explanatory gain.** The evaluation size is fixed exactly: $m(n) = n^2$ outcome bits, at fresh input locations that are pairwise distinct and were never queried during discovery, drawn after discovery closes. On that evaluation set, the generating program earns $\Omega(n^2)$ Turings relative to the uniform baseline, a gain density bounded away from zero.
4. **Persistent discovery barrier.** For every probabilistic polynomial-time learner, the probability of producing a model that earns at least a fixed fraction of the available gain tends to zero as n grows. The probability is taken over the random draw of the hidden world, the observations, the evaluation inputs and outcomes, and the learner's randomness.

I propose 10% as the fraction. Its exact value is secondary to the existence of a separation that persists as the problems grow.

The learner may produce **any effective predictor**; recovering the original generating program is unnecessary.

Conditions for a meaningful test

The baseline, encoding, sampling process, permitted experiments, and evaluation protocol must be fixed in advance. Evaluation outcomes must remain unavailable during discovery, and evaluation inputs must be fresh and pairwise distinct, never queried during discovery. Every submitted model must pay its description cost, including anything its decoder uses beyond the input locations.

The observation and experiment budget must be specified alongside the computational budget and the evaluation size. These choices are part of the conjecture: changing them may change whether the claim holds.

Costs are kept in two separate accounts throughout. Description costs are measured in bits and charged in the Turing accounting. Operating costs, including search time, program maintenance, and library construction, are measured as computation and tracked under the discovery budget. No conversion between the two accounts is used unless a conversion rule is explicitly defined; I define none here.

Three claims, three standards

Claim A: a conditional result, proved. I prove that under the assumption that one-way functions exist, a family satisfying the four conditions exists, with the baseline fixed to uniform on the evaluation outcomes. The route runs through pseudorandom functions: compact keys, efficient evaluation, outputs that resist efficient prediction [5]. The separation phenomenon has precedent in the cryptographic limits on learning established by Kearns and Valiant [6]. Appendix C gives the construction, the sampling rules, and the full proof: a learner beating the barrier would distinguish the pseudorandom function from a truly random one, contradicting its security. The proof is carried out under AIEN’s accounting rules, including the decoder rule, the efficient-coding requirement, and the gain density.

Claim B: an unconditional conjecture, to investigate. The same separation holds with no cryptographic assumption. I have not established a reduction between this claim and the existence of one-way functions, and I do not assert one. Some precisely defined hardness results for time-bounded Kolmogorov complexity are equivalent to their existence [9]; whether anything comparable holds for this formulation is itself an open question. Claim B touches deep open questions in cryptography and complexity theory, and is stated at exactly that strength, no stronger.

Claim C: an AIEN program-reuse hypothesis, to test. Stated as a protocol, so that it can fail:

- *Task family.* A sequence of stochastic-law discovery tasks drawn from one declared generator family whose laws share components, of the kind used in AIEN’s sealed-evaluation experiments [12, 13]: a hidden law, observations during discovery, and a sealed held-out evaluation under a frozen profile.
- *Library-building procedure.* After each task, any program that earned positive net Turings on that task’s sealed evaluation is admitted to the library. Admission charges the program’s description length to the library, and every admitted program is part of the model, charged under the decoder rule, whenever the library is used.
- *Comparison.* Two learners run the same task sequence under equal candidate-evaluation allowances: 20,000 candidate evaluations per task per learner, one with access to the growing

library, one with an empty library throughout. The allowance covers every candidate scored against context data, including library admission checks and library use. Wall-clock time and measured joules are recorded as outcomes, not equalized: the protocol guarantees equal evaluation counts, not equal consumed computation.

- *Hypothesis.* The library learner’s total net Turings across the sequence, with all library description costs charged, exceed the library-free learner’s total under equal candidate-evaluation allowances. Per-task amortization of library costs may also be reported, but the headline figure is the sequence total, with construction costs included, never waived.

Lifelong learning and library learning research have explored versions of this idea [2, 4, 14]; the specific, measured form stated here, with library costs charged in bits and construction counted in the budget, is mine to demonstrate. One boundary is firm: a library independent of a fresh secret key does not defeat a secure pseudorandom function. Reuse helps when tasks share structure. It is not a universal solvent for discovery barriers. Appendix D states the experiment as a draft preregistration specification, with the remaining values to be settled and committed before the first sequence runs.

What would resolve it?

I prove Claim A in Appendix C under the standard assumption that one-way functions exist. A proof of Claim B would establish it outright. A disproof of Claim B would establish that no family satisfies all four conditions under the specified protocol. Efficiently learning one difficult family would not, by itself, disprove an existence claim.

Experiments bear directly on Claim C and can expose weaknesses in the formulation of Claims A and B. They cannot establish the universal claims.

Significance for AIEN

In this conjecture I distinguish **available explanatory value** from **accessible explanatory value**.

Turings measure the gain an explanation delivers. They do not automatically measure the work required to discover it.

For AIEN, I take the foundational question to be Claim C: when do retained programs, accumulated knowledge, and reusable explanations make discovery measurably cheaper, in Turings gained per unit of discovery budget, and when does a computational barrier remain?

Research status

I propose this as a conjecture, not an established theorem or a recognized Millennium Prize problem. Its conditional part is proved in Appendix C by reduction to pseudorandom-function security; its unconditional part is open and not known by me to be equivalent to any standard assumption; its program-reuse part is an empirical hypothesis with named predecessors, now stated as a falsifiable protocol with a draft preregistration experiment specification. Quantitative separations between explanations that exist and explanations efficient algorithms can find already appear in the literature; naming the unit does not by itself constitute a mathematical contribution. What

I offer here is a proof under AIEN’s accounting rules and a working measurement protocol. The program-reuse advantage of Claim C is an untested hypothesis: CEXP-1 has not run.

A world may be easy to explain once its rule is known, yet fundamentally hard to learn.

Acknowledgments and provenance

The proof technique of Appendix C, the counting lemma and the bounded sampler, follows an independent draft by GPT-6 Astra, shared with me on 2026-09-30; I have restated both in this document’s notation and protocol. The problem statement, the accounting rules, the draft CEXP-1 preregistration specification, and the references are my own. The proof as written stands on its own derivation; this note records provenance, not a dependency.

A Valid compression

A submission is scored only if its code is valid, in the following sense.

- **Prefix-free codes.** All description lengths are lengths under a prefix-free binary code: no codeword is a prefix of another. By the Kraft-McMillan theorem, a set of lengths is realizable by a uniquely decodable code exactly when the corresponding Kraft sum satisfies $\sum_i 2^{-\ell_i} \leq 1$ [3, 11]. Unique decodability is what makes a length an honest count of information.
- **Lossless decoding.** Decoding the submitted encoding, given the input locations X and the charged model, must reproduce the evaluation outcomes exactly. A short output that does not decode to the actual outcomes is not compression; it is an error.
- **Invalid submissions earn no score.** If decoding fails, is lossy, or runs outside the polynomial time bound of the efficient-coding rule, the submission is rejected and earns no Turings. There is no partial credit for approximate reconstruction; approximation belongs in the model’s predictions, where its cost is paid openly in the conditional description length.
- **Uniform scoring bound.** The profile fixes one polynomial step budget for the whole evaluation: encoding, decoding, and reconstruction checking, including the overhead of the simulation that runs the submitted code. The scorer executes submitted code under that budget and rejects timeouts or failed reconstructions. It does not attempt to decide the asymptotic runtime bounds of arbitrary programs; it only enforces the declared budget. The key-holding reference program fits within the chosen budget. This is what the Claim A proof relies on when it scores a submitted model: the reliance is on the scorer, stated here as a requirement.
- **One code, fixed in advance.** The encoding scheme, and therefore the meaning of L , is part of the frozen profile. It cannot be chosen after seeing the evaluation data.

B Complete probability model

A learning problem in the family is fully specified by the following, all fixed in advance:

1. **World sampling.** A declared, efficiently samplable distribution over generating programs of the family at size n . The hidden world is one draw from this distribution.

2. **Input space and input distribution.** A declared set of input locations and a declared, efficiently samplable distribution over them. Evaluation inputs are drawn from this distribution, conditioned on never having been queried during discovery and on being pairwise distinct.
3. **Discovery interaction.** During discovery the learner may adaptively query input locations of its choice and receive the true outcomes, up to a declared query budget, in the style of query learning [1, 15], and may also receive observations drawn from the input distribution, up to a declared observation budget. Both budgets are polynomial in n and are part of the problem specification.
4. **Freeze point.** When the budgets are exhausted, the learner outputs one model and is frozen. No further queries, updates, or selection among models is permitted after the freeze. Model selection performed during discovery is part of the learner and is charged to its computation budget.
5. **Evaluation draw.** After the freeze, $m(n) = n^2$ fresh, pairwise distinct input locations are drawn, their outcomes computed by the hidden generating program, and the frozen model is scored exactly once under the accounting rules.
6. **Success and failure.** The available gain is the frozen evaluation score of the generating program itself. A learner succeeds if its score is at least the declared fraction (proposed: 10%) of the available gain. Claims A and B assert that, over the draws in steps 1, 2, and 5 and the learner's own randomness, the success probability of every probabilistic polynomial-time learner tends to zero as n grows.

C Instantiation for Claim A: the pseudorandom-function family

The conditional construction, with every sampling rule made explicit:

- **World sampling.** Draw a key k uniformly at random from $\{0, 1\}^n$. The generating program is the pseudorandom function F_k of Goldreich, Goldwasser, and Micali [5], built from a pseudorandom generator, which exists if one-way functions exist. The program consists of the key plus a fixed evaluator: description length $O(n)$, evaluation in time polynomial in n .
- **Input space and distribution.** Inputs are n -bit strings, $x \in \{0, 1\}^n$, so the domain has 2^n locations, large enough that exhaustion and collision are not strategies. The input distribution is uniform, which is efficiently samplable.
- **Discovery interaction.** The learner makes adaptively chosen input queries and receives $F_k(x)$ for each, up to a query budget of $q(n) = n^3$. Let Q be the set of all inputs whose outcomes were revealed during discovery, by query or by observation. Then $|Q| \leq n^3$, negligible against 2^n .
- **Evaluation draw.** Draw $m(n) = n^2$ evaluation inputs uniformly without replacement from $\{0, 1\}^n \setminus Q$. The evaluation inputs are therefore pairwise distinct and disjoint from everything the learner has seen. Distinctness is load-bearing: a repeated input would let any model earn from repetition itself, and an input repeated from discovery would let a charged transcript answer it, so compressible repetition could appear even though the function resists discovery. Outcomes are the bits $F_k(x)$, one per input, for n^2 outcome bits in total.

- **Baseline.** The baseline is uniform on the evaluation outcomes: each bit is assigned probability one half. The uniform predictor needs no parameters, so $L(B)$ is a fixed constant, and $L(D \mid B, X) = m$ for every D .
- **Available gain.** The key-holding program predicts every evaluation bit. Under the frozen code its score is the baseline cost of n^2 near-random bits, minus its $O(n)$ description and a small conditional cost: $T(P_k; B, D, G, P) = n^2 - n + O(1)$, hence at least $n^2/2$ Turings for all large n , an $\Omega(n^2)$ gain with density approaching 1, bounded away from zero.
- **Barrier: proof of Claim A.** Suppose, for contradiction, that some probabilistic polynomial-time learner A violates Condition 4: for some constant $c > 0$ and infinitely many n , A earns at least one tenth of the available gain with probability at least c , the probability taken over the key draw, the discovery interaction, the evaluation draw, and A 's randomness. For large n the available gain is at least $n^2/2$, so success implies a Turing score of at least $n^2/20$. The argument works for any fixed positive fraction, and for any polynomial discovery budget, not only n^3 .

One lemma does the work.

Counting lemma. Fix the discovery transcript, the resulting frozen model M , and the evaluation input list X . Assume the baseline B is uniform on the evaluation outcomes, so that $L(D \mid B, X) = m$ for every D . If the m evaluation outcome bits are independent uniform bits conditional on those fixed objects, then for any threshold s ,

$$\Pr[T(M; B, D, G, P) \geq s \mid \text{transcript}, M, X] \leq 2^{L(B)-s}.$$

Proof of lemma. By the uniform-baseline hypothesis, scoring at least s requires $L(D \mid M, X) \leq L(B) + m - L(M) - s$. Call that bound r ; if $r < 0$ the event is impossible. The residual code is prefix-free, so by the Kraft inequality it contains at most 2^r codewords of length at most r , and each codeword decodes to at most one outcome string D . Of the 2^m equally likely outcome strings, at most 2^r can reach the threshold, so the probability is at most $2^{r-m} = 2^{L(B)-L(M)-s} \leq 2^{L(B)-s}$. The bound holds even for a randomized encoder: it counts decodable short residuals, not encoder behavior. Averaging over transcripts, models, and input lists preserves it. $\square$

Now construct a probabilistic polynomial-time distinguisher D with oracle access to $O : \{0, 1\}^n \rightarrow \{0, 1\}$:

1. Simulate discovery: run A , answering its adaptive queries with O ; let Q be the set of all revealed inputs.
2. Let M be the frozen model A outputs.
3. Simulate the evaluation draw with a bounded sampler: repeatedly draw uniform inputs, accepting only points outside Q not already accepted, until n^2 inputs are accepted or $2n^2$ attempts are made; on failure, output 0. Query O for the accepted inputs' outcome bits and compute M 's Turing score under the frozen profile and baseline. Output 1 if the score is at least $n^2/20$, else 0.

D runs in polynomial time: A does, it makes polynomially many oracle queries, and scoring a valid model is polynomial by the efficient-coding rule. An invalid model scores nothing. The bounded sampler keeps D strictly polynomial rather than relying on unbounded rejection sampling. Its failure probability is at most $4^{n^2} a^{n^2}$ with $a = (|Q| + 2n^2)/2^n$: failure needs at

least n^2 rejected attempts among $2n^2$, each attempt rejects with conditional probability at most a , and a union bound over the choices of the rejected positions gives the bound. The exact count is the binomial coefficient $\binom{2n^2}{n^2}$, and 4^{n^2} is a valid but loose upper bound for it; the looseness is harmless, since only negligibility is needed. Since $|Q|$ is polynomial, a is far below $1/4$ for large n and the failure probability is negligible; failure only lowers acceptance.

If $O = F_k$ for a uniform key k , the simulation reproduces the Claim A world, and D 's evaluation draw has the game's distribution whenever the sampler completes, so D outputs 1 with probability at least $c - \text{negl}(n)$ on the infinite subsequence.

If $O = R$ is a truly random function, the outcome bits at the fresh accepted points are independent uniform bits even after conditioning on the adaptive discovery transcript and the accepted input list, so the counting lemma bounds D 's acceptance probability by $2^{L(B)-n^2/20}$. The baseline length $L(B)$ is a constant, so this is negligible in n .

The distinguishing advantage is therefore at least $c - \text{negl}(n)$, bounded away from zero on the infinite subsequence. This contradicts the pseudorandomness of the Goldreich-Goldwasser-Micali family [5], which holds if one-way functions exist. Hence no such learner A exists: every probabilistic polynomial-time learner's success probability tends to zero. That is Condition 4, and Claim A is proved. $\square$

D The Claim C experiment, draft preregistration specification (CEXP-1)

- Apparatus and implementation identity.** The AIEN stochastic-law apparatus in its EXP-002 "Stochastic Law Discovery" configuration: candidates compete to explain sealed held-out trajectories from a freshly generated stochastic law [12, 13]. Measurement profile: turing-profile v1.0 (draft; any change is a new version). Coder: the profile's reference coder, theoretical description length under declared predictive distributions. Quantizer Q : the scored quantity is the one-step increment $\Delta_t = x_t - x_{t-1}$, not the absolute position: absolute positions of a diffusion wander without bound, so a fixed position grid piles mass into its edge bins. Increments are quantized into 256 uniform bins over $[-5, 5]$; out-of-range increments are clipped to the edge bins. The generator family declares its maximum per-step diffusion coefficient in the preregistration manifest, and a run is valid only if the clipped fraction is reported and stays at or below 5.0% of sealed observations. If more than 5.0% of sealed observations fall outside $[-5, 5]$ before quantization and are therefore clipped, the affected preregistered condition is invalid for the primary claim; if CEXP-1 has only one primary condition, that invalidates the run. The failed run is preserved and reported, not deleted and rescored. Clipping is measured on actual out-of-range values before quantization, not on edge-bin occupancy: raw pre-quantization values are retained so the gate uses the true clipped fraction, with edge-bin occupancy reported as a diagnostic. Baseline: uniform over quantization bins, 8 bits per scored increment. Model cost convention: 32 bits per fitted real parameter, 8 bits per small integer hyperparameter selected on context data.
- Generator configuration.** Each task's law is freshly generated by the declared generator family with a sealed seed. For Claim C the family is configured so that laws are composed from a shared component pool, so successive tasks genuinely share structure; the component pool and composition rule are written into the preregistration manifest before the first sequence runs.

- **Task and sequence structure.** A sequence is 12 tasks. Per task, discovery receives 8 context trajectories of 400 observations each. Evaluation is one-step-ahead prediction on 4 sealed trajectories of 400 observations, scoring predictions at steps 1 to 399 of each, for 1,596 scored observations per task.
- **Learner settings.** One discovery procedure, identical hyperparameters in both configurations. The library learner retains only programs admitted under Claim C’s admission rule (positive net Turings on the task’s sealed evaluation; description length charged). The comparison learner retains nothing across tasks. Both learners see identical task sequences, paired by seed.
- **Budgets.** Per task and per learner, the discovery budget is 20,000 candidate evaluations, where one candidate evaluation is one candidate scored against the task’s context data. Library admission checks and any library use are candidate evaluations and count against the same allowance. Observation access is exactly the context trajectories above; there are no additional queries. The run executes on one explicitly identified DGX Spark (GB10) machine configuration, exclusively, with identical execution limits for both arms. Wall-clock time and measured joules are recorded outcomes, not stopping rules: the protocol equalizes the number of candidate evaluations each learner may spend, and the physical costs each learner actually consumed are reported alongside the scores, not assumed equal. Library construction, admission, and maintenance cost real time and energy, and those costs fall on the library learner. A runaway ceiling guards against unbounded runs: the wall-time and energy abort ceilings are a fixed multiple of the measured worst-case development-seed cost from an unsealed calibration run, and the resulting exact hours and joules are written into the preregistration manifest. A ceiling is an abort rule, not an equalizer of consumed computation: a ceiling violation aborts the learner, it does not grant more evaluations.
- **Replication.** 30 paired sequences, the two configurations run on identical seeds and identical task draws.
- **Criterion and uncertainty analysis.** The primary criterion is the expected total net Turings across the 12-task sequence, all library costs charged. The effect is estimated by the mean paired difference of sequence totals across the 30 replications, with a 95 percent confidence interval from a percentile bootstrap over sequences (10,000 resamples). The hypothesis counts as supported only if the interval lies entirely above zero; if it straddles zero the result is reported as inconclusive, not as support. The full distribution of paired differences is reported alongside the mean.
- **Preregistration.** This specification is a draft until four items are settled and committed before the first sequence runs: the runaway-ceiling multiplier and the resulting wall-time and energy limits, the treatment of a learner that aborts at the ceiling, the sequence-level library-cost ledger formula, and the encoding costs for program structure and component selection. With those committed, the specification, the generator configuration, and the analysis method are frozen before the first sequence runs. Deviations void the run’s claim to test Claim C.

References

- [1] Angluin, D. (1988). Queries and concept learning. *Machine Learning*, 2(4), 319-342.
- [2] Chen, Z., and Liu, B. (2018). *Lifelong Machine Learning* (2nd ed.). Morgan and Claypool.

- [3] Cover, T. M., and Thomas, J. A. (2006). *Elements of Information Theory* (2nd ed.). Wiley.
- [4] Ellis, K., Wong, C., Nye, M., Sablé-Meyer, M., Morales, L., Hewitt, L., Cary, L., Solar-Lezama, A., and Tenenbaum, J. B. (2021). DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. *Proceedings of PLDI 2021*, 835-850.
- [5] Goldreich, O., Goldwasser, S., and Micali, S. (1986). How to construct random functions. *Journal of the ACM*, 33(4), 792-807.
- [6] Kearns, M. J., and Valiant, L. G. (1994). Cryptographic limitations on learning Boolean formulae and finite automata. *Journal of the ACM*, 41(1), 67-95.
- [7] Kolmogorov, A. N. (1965). Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1), 1-7.
- [8] Li, M., and Vitányi, P. (2019). *An Introduction to Kolmogorov Complexity and Its Applications* (4th ed.). Springer.
- [9] Liu, Y., and Pass, R. (2020). On one-way functions and Kolmogorov complexity. *Proceedings of FOCS 2020*, 1243-1254.
- [10] Rissanen, J. (1978). Modeling by shortest data description. *Automatica*, 14(5), 465-471.
- [11] Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379-423, and 27(4), 623-656.
- [12] Stapleton, M. D. (2026). *Computing Machinery and Understanding: From the Imitation Game to the Turing*. AIEN Project, v1.13. aienos.com/research.
- [13] Stapleton, M. D. (2026). *The Turing as a Measure Emerging from AIEN*. AIEN Project, v1.7. aienos.com/research.
- [14] Thrun, S., and Mitchell, T. M. (1995). Lifelong learning: A dialogue on situated and integrated learning agents. In *Working Notes of the AAAI Spring Symposium on Integrated Learning Architectures*.
- [15] Valiant, L. G. (1984). A theory of the learnable. *Communications of the ACM*, 27(11), 1134-1142.